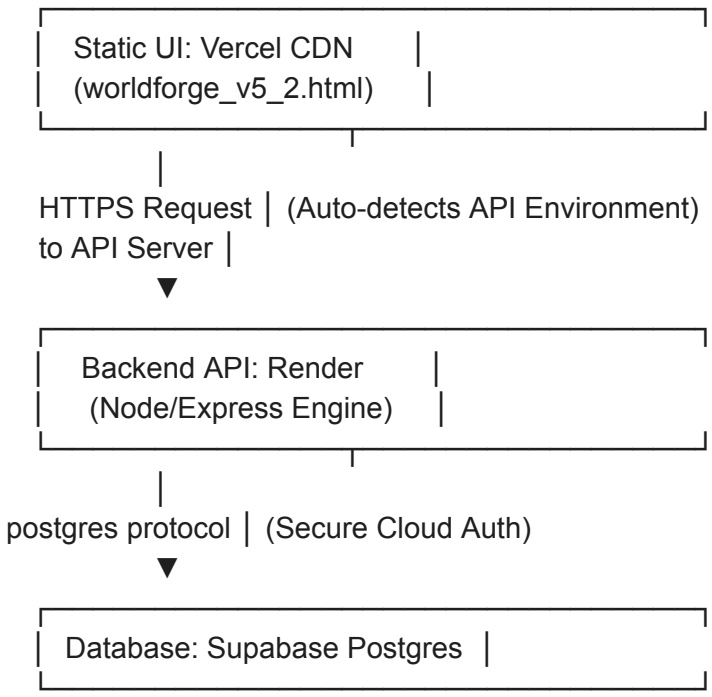
System Blueprint: Phase 2 Frontend-Backend Integration

Version 2.0 — Architecture Specification for Unified Interface & Deployment

This specification defines the integration pathways to connect our local single-file UI (worldforge_v5_2.html) to our live Node/Express API backend and Supabase database. It details the zero-terminal deployment strategy to push both services live on the web for free.

1. Integration Topology

We maintain a separation of concerns while running a zero-dollar staging environment. The client UI is hosted statically on a global CDN, communicating with our backend server over CORS-enabled REST endpoints.



2. Environment Auto-Discovery Protocol

To prevent hardcoding development URLs during deployment, the client UI must automatically toggle its active API target based on where the browser window is loaded.

- **Localhost Execution:** If loaded from file:/// or http://localhost, target the local development backend (http://localhost:5000).
- **Production Execution:** If loaded from our Vercel CDN domain, automatically target our

live production Render backend service.

```
// Base API configuration within UI
const API_BASE = window.location.hostname === 'localhost' || window.location.protocol === 'file:'
    ? 'http://localhost:5000'
    : 'https://your-deployed-render-service.onrender.com';
```

3. Zero-Terminal Deployment Strategies

We leverage visual Git integration and managed SaaS platforms to publish our operating ecosystem with zero-terminal compilation commands.

A. Database Provisioning (Supabase GUI)

1. Log into your **Supabase Dashboard** in your browser.
2. Select your project, open the **SQL Editor** from the left-hand navigation pane.
3. Copy-paste the entire database schema code block from /docs/specs/blueprint_v1_agent_foundry.md directly into the editor and click **Run**.
4. Navigate to **Project Settings -> API** to retrieve your Project API URL and anon public key.

B. Backend Deployment (Render.com)

1. Register for a free account on **Render.com** and select **New + -> Web Service**.
2. Connect your GitHub account and select your worldforge-os repository.
3. Configure the following runtime variables on the dashboard:
    - **Runtime:** Node
    - **Build Command:** npm install
    - **Start Command:** node src/server.js
4. Expand the **Advanced / Environment Variables** pane and add:
    - SUPABASE_URL = *[Your Supabase Project URL]*
    - SUPABASE_ANON_KEY = *[Your Supabase Anonymous Key]*
    - PORT = 10000 *(Render provides this port automatically)*
5. Click **Create Web Service**. Render will spin up the environment and provide an HTTPS server link.

C. Frontend Deployment (Vercel CDN)

1. Sign in to **Vercel.com** with your GitHub credentials.
2. Click **Add New -> Project**, then import your worldforge-os repository.
3. In the project setup panel:
    - **Framework Preset:** Other (or static HTML)
    - Keep the default root directory.
4. Click **Deploy**. Vercel will process the HTML file and provide a public, optimized web URL.

4. Next Steps for Claude (Automated Execution Prompt)

*"Claude, read /docs/specs/blueprint_v2_integration.md. Using your filesystem tool, update our frontend visual client (worldforge_v5_2.html) to establish active database-backed operational status:

1. Replace the static dummy data ROSTER_DATA inside the script with an automatic asynchronous initialization function (fetchRoster) that calls /api/roster.
2. Add environment auto-discovery logic so that API calls point to http://localhost:5000 when running locally, and our upcoming Render domain when deployed.
3. Update the 'Build Structure' submit handler and the 'Optimize' function to POST the newly generated configurations back to our database using /api/agents to save updates persistently."*